INFORME DE CALIDAD SPRINT 02

- Grupo: 1
- Sprint: 2
- Fecha de realización: 04/11/2026
- Commit sobre el que se realiza el análisis: 89db6cd

1. Capturas de SonarCloud y ScrumDesk:

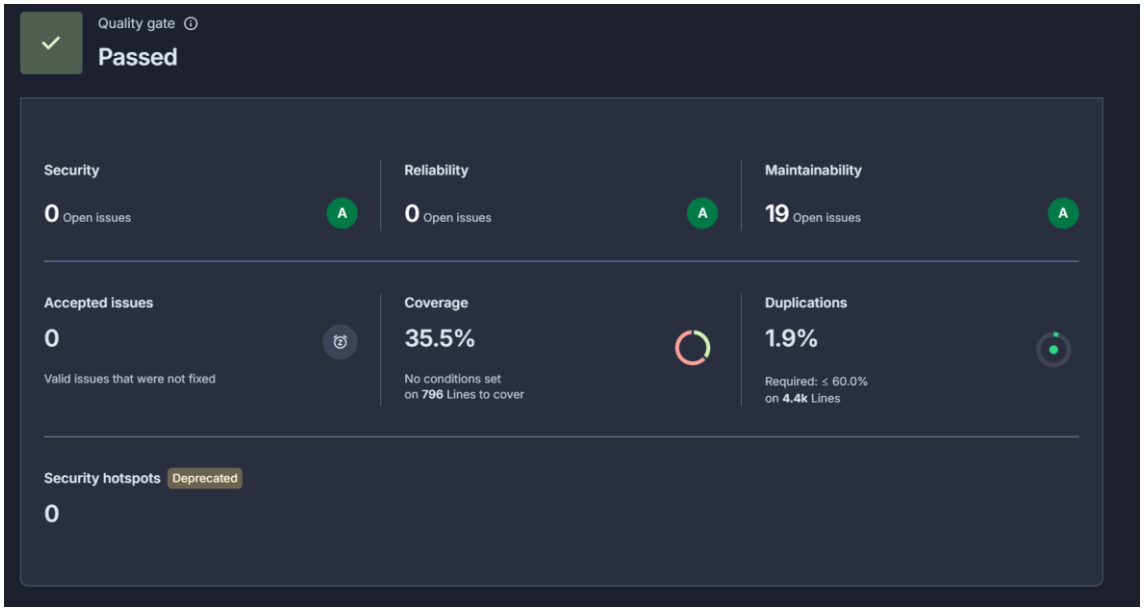


Imagen 1. Resumen del análisis

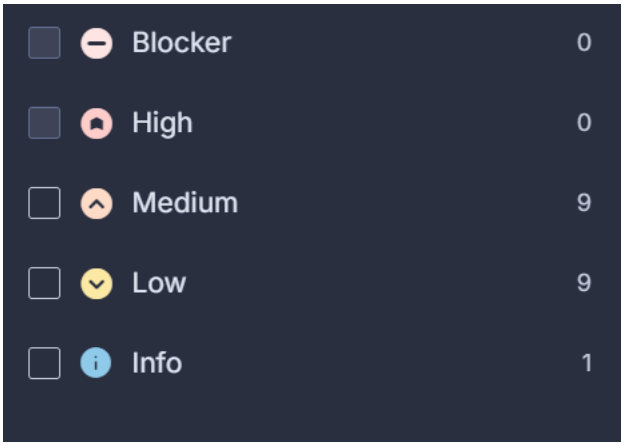


Imagen 2. Severidad de los issues de Maintainability

Nota: Al no haber issues de Security ni de Reliability, se omiten las imágenes con la severidad de dichos issues.



Imagen 3. Gráfica de número de issues de cada uno de los elementos



Imagen 4. Gráfica de porcentaje de código repetido

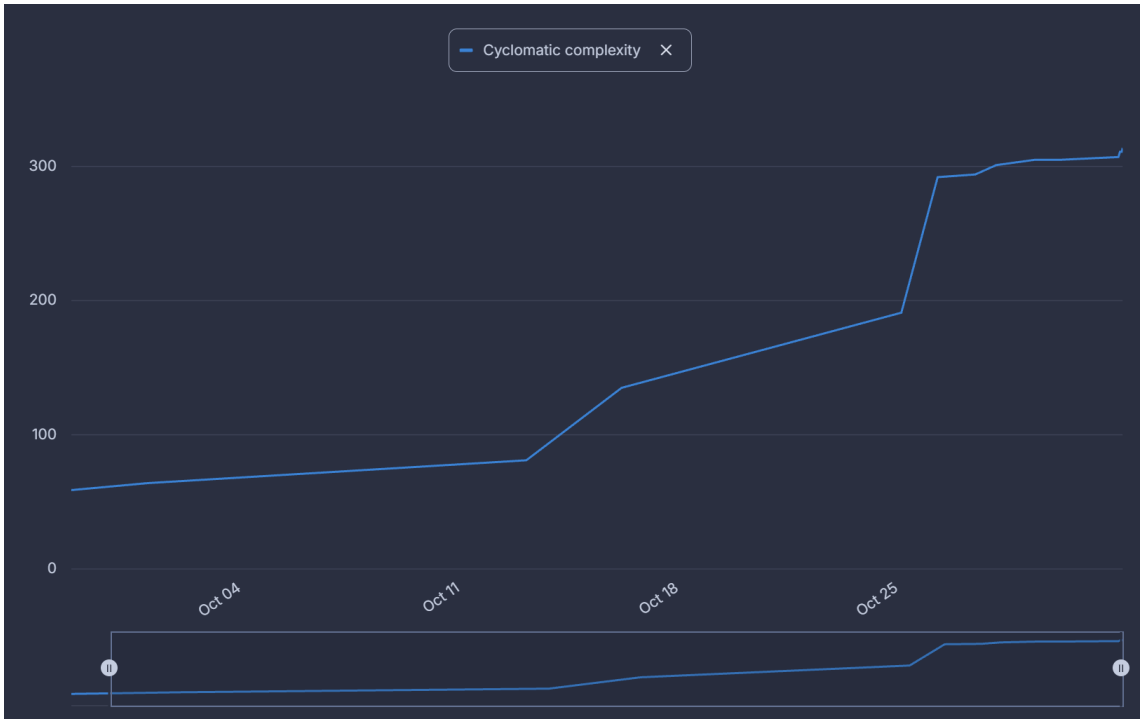


Imagen 5. Gráfica de complejidad ciclomática

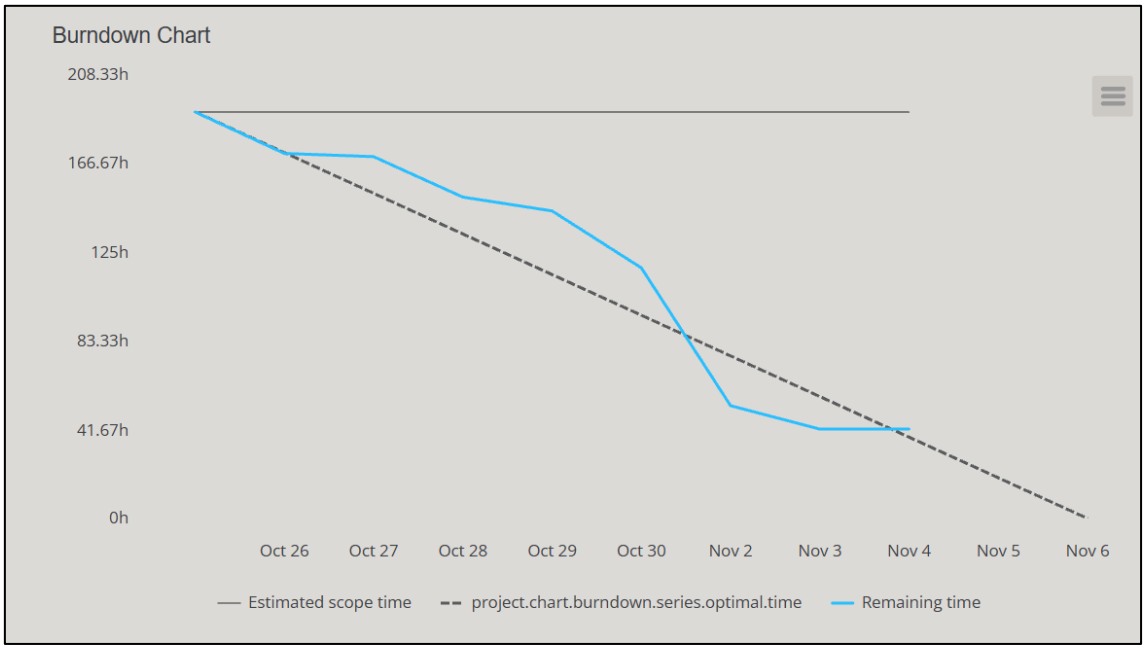


Imagen 6. Burndown Chart

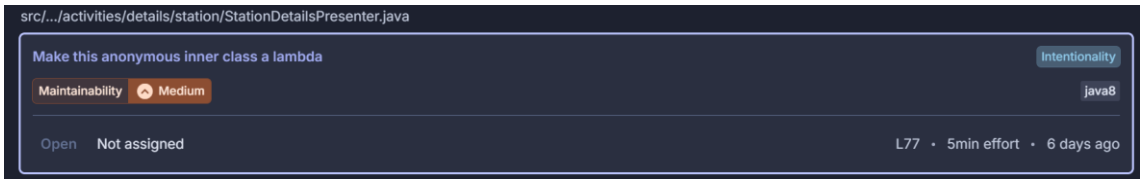


Imagen 7. Issue a solucionar en la clase *StationDetailsPresenter*

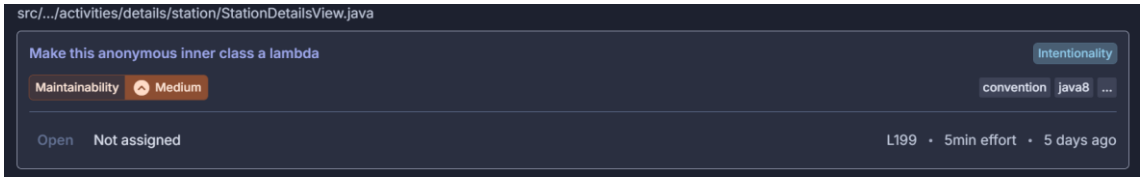


Imagen 8. Issue a solucionar en la clase *StationDetailsView*



Imagen 9. Issue a solucionar en la clase *StationDetailsView*

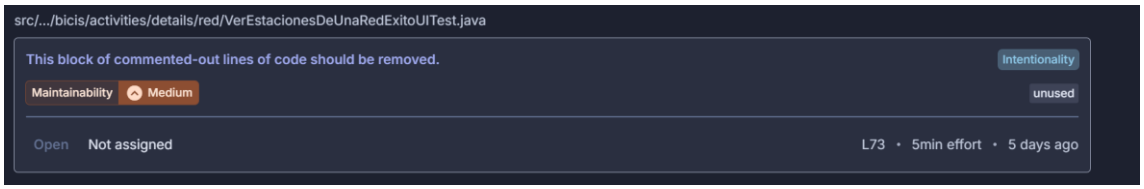


Imagen 10. Issue a solucionar en la clase *VerEstacionesDeUnaRedExitoUITest*



Imagen 11. Issue a solucionar en el fichero *themes.xml*

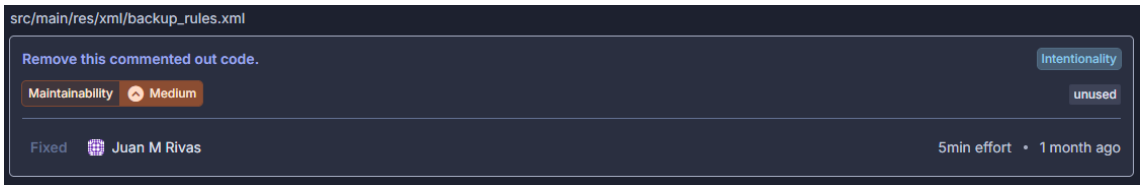


Imagen 12. Issue a solucionar en el fichero *backup_rules.xml*



Imagen 13. Issues a solucionar en la clase *IncidentArrayAdapter*

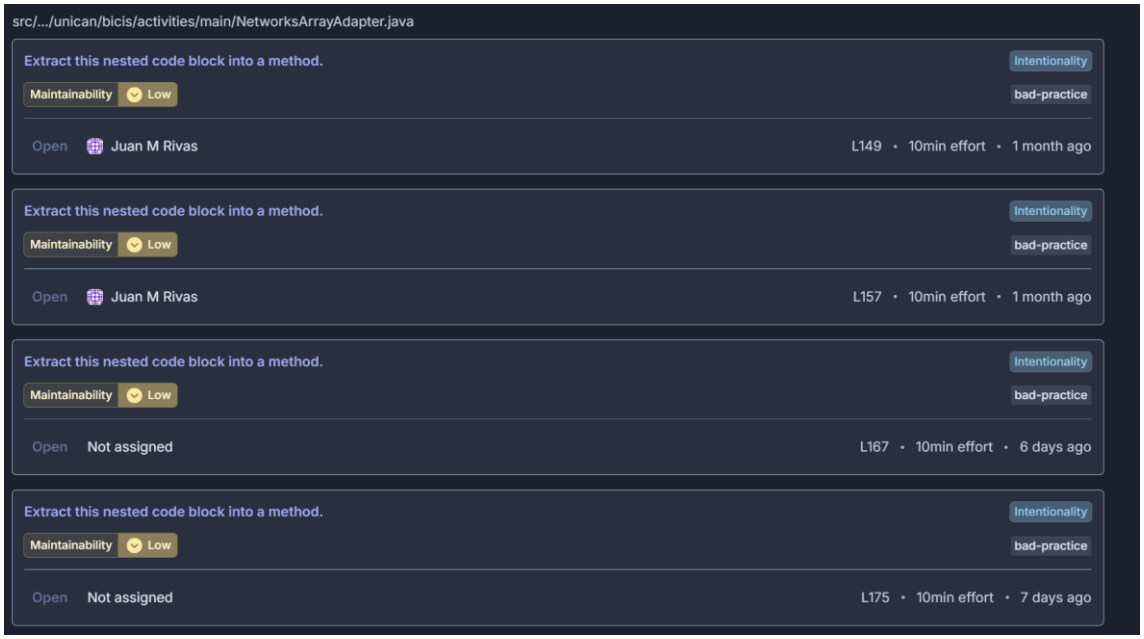


Imagen 14. Issues a solucionar en la clase *NetworksArrayAdapter*

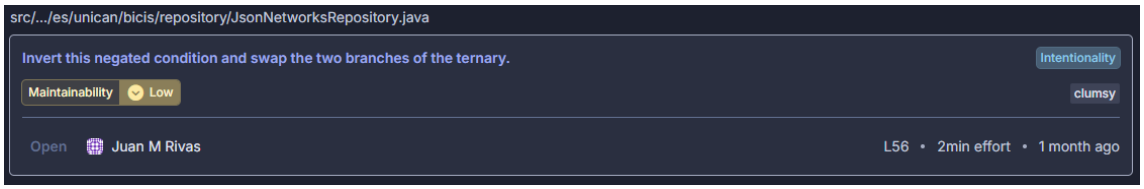


Imagen 15. Issue a solucionar en la clase *JsonNetworksRepository*



Imagen 16. Issue a solucionar en el fichero *data_extraction_rules.xml*

2. Análisis del estado del producto:

En primer lugar, como se puede observar en la Imagen 1, el Quality Gate para el análisis pasa satisfactoriamente. Además, tan solo existen 19 issues de mantenibilidad, cuyas severidades no son preocupantes, 9 de ellos media, 9 baja, y 1 de tipo info (Imagen 2). El effort de estos issues es bajo en general, variando entre 1 y 10 minutos, y dando lugar a una deuda técnica general de 1 hora y 59 minutos.

En este sentido, como se ve en la Imagen 3, los issues de mantenibilidad han aumentado considerablemente desde el inicio del sprint, disminuyendo de forma notable en el tramo final, a causa de diferentes tareas de mantenimiento que fueron llevadas a cabo. También durante el sprint se introducen algunos issues de fiabilidad, que desaparecen por completo antes del análisis. Algo similar ocurre con el porcentaje de código repetido, que crece desde el último análisis, y de forma especialmente pronunciada desde el inicio del segundo sprint, pasando de un 0% del análisis del sprint 1 a un 1,9%, como se observa en la Imagen 1 y en la Imagen 4. No obstante, debido al mantenimiento realizado, el código duplicado disminuye de forma notable conforme avanza este sprint. En cuanto a la complejidad ciclomática (Imagen 5), esta aumenta alrededor de 110 puntos durante el sprint, de forma muy pronunciada al inicio de este, lo cual se justifica debido a la incorporación del código que dará soporte a las nuevas funcionalidades.

En resumen, el estado del producto en términos de calidad es muy positivo, el número de issues es bajo y su severidad no es preocupante. Además, la deuda técnica es también reducida y parámetros como el porcentaje de código duplicado o la complejidad ciclomática, aunque crecen, no lo hacen de forma preocupante y se mantienen controlados, más aún sabiendo que estos ligeros incrementos surgen de la incorporación de nuevo código, necesario para implementar las funcionalidades requeridas.

Finalmente, como se puede observar en la Imagen 6, el tiempo restante estimado del trabajo por realizar en el sprint es levemente superior al óptimo. Sin embargo, este dato puede resultar algo engañoso, ya que como muestra la gráfica, en los días previos este tiempo restante se encontraba notablemente por debajo del óptimo y la tendencia era muy positiva, lo cual indica que el ritmo del equipo de trabajo es bueno y que el margen de tiempo es amplio. No obstante, en el momento de la realización de este informe, hay varias tareas en progreso que están a punto de ser terminadas, lo cual liberará a ciertos miembros del equipo, y hará que el tiempo restante estimado vuelva a ser inferior al óptimo, lo cual demuestra un buen margen temporal.

3. Plan de acción:

Como se ha explicado en el análisis del estado del producto, la calidad de este es alta, y no existen issues que deban ser corregidos urgentemente. Sin embargo, el estado del sprint es positivo en cuanto a trabajo por realizar, y se dispone de un ligero margen de tiempo extra. Por este motivo, y sabiendo (más adelante se detalla) que de los 19 issues, hay 3 que se encuentran duplicados, y que la gran mayoría se corresponden con un mismo issue en diferentes partes del código, se propone un plan de acción con el objetivo de mejorar la calidad del producto y reducir la deuda técnica de cara a futuros sprints.

En concreto, se destinarán 2 horas a la aplicación de dicho plan, que se asocia a la tarea creada en ScrumDesk "Aplicar plan de mejora". En el plan se listan y priorizan los issues que se deberán solucionar. Como todos los issues son de mantenibilidad, se prioriza dar solución a aquellos con mayor severidad, pues así se conseguirá aumentar la calidad lo máximo posible en el tiempo del que se dispone. Ante issues de la misma severidad, aquellos que se repitan en diferentes partes, tendrán la misma prioridad, si son diferentes, se priorizan los que se consideren que su corrección suponga un mayor beneficio para el código.

El plan de acción es el siguiente:

1. Convertir clase interna anónima en una lambda en la clase *StationDetailsPresenter*. Este issue se encuentra duplicado, por algún motivo, en la herramienta SonarCloud. Supone un effort de 10 minutos (en realidad es de 5, pero al estar duplicado es de 10). Ver Imagen 7.
2. Convertir clases internas anónimas en lambdas en la clase *StationDetailsView*. Este issue aparece dos veces en la clase, y al igual que el anterior, ambos se encuentran duplicados. Supone un effort de 10 minutos cada uno. Ver Imagen 8 e Imagen 9.
3. Eliminar bloque de código comentado de la clase *VerEstacionesDeUnaRedExitoUITest*, con un effort de 5 minutos. Ver Imagen 10.
4. Eliminar bloque de código comentado en fichero *themes.xml*, que estima un effort de 5 minutos. Ver Imagen 11.
5. Eliminar bloque de código comentado en fichero *backup_rules.xml*, con effort estimado de 5 minutos. Ver Imagen 12.
6. Renombrar el atributo *Incidences*, para que cumpla las reglas de nomenclatura de Java, en la clase *IncidentArrayAdapter*. Supone un effort de 2 minutos. Ver Imagen 13.
7. Extraer bloques de código a métodos en la clase *IncidentArrayAdapter*. El issue se repite tres veces en la clase y cada uno tiene un effort de 10 minutos. Ver Imagen 13.
8. Extraer bloques de código a métodos en la clase *NetworksArrayAdapter*. El issue aparece cuatro veces en la clase y cada uno supone un effort de 10 minutos. Ver Imagen 14.
9. Invertir condición negativa en operador ternario en la clase *JsonNetworksRepository*, lo cual supone un effort de 2 minutos. Ver Imagen 15.
10. Completar tarea asociada al comentario "TODO" en *data_extraction_rules.xml*, con un effort estimado de 0. Ver Imagen 16.

En total, se prevé dar solución a los 19 issues, reduciendo tanto estos, como la deuda técnica, a 0, lo cual supone un effort total de 1 hora y 59 minutos, tiempo dentro del destinado al plan.